

MANUAL GERAL DE ORGANIZAÇÃO E DESENVOLVIMENTO

Marketplace Comunitário Raízes do Araguaia

1. Objetivo do Projeto

O Marketplace Comunitário Raízes do Araguaia é uma plataforma desenvolvida para conectar comunidade, produtos, educação, eventos e geolocalização em uma experiência digital acessível, organizada e preparada para crescimento futuro.

Este manual define os padrões oficiais de organização, desenvolvimento, comunicação e entrega da equipe.

Todos os membros devem seguir este documento como referência principal durante o desenvolvimento do projeto.

2. Objetivo da Organização da Equipe

A estrutura organizacional foi criada para:

- Evitar retrabalho.
 - Reduzir conflitos no GitHub.
 - Melhorar a comunicação entre membros.
 - Padronizar desenvolvimento.
 - Organizar dependências técnicas.
 - Facilitar integração entre módulos.
 - Garantir previsibilidade nas entregas.
 - Facilitar onboarding de novos integrantes.
 - Permitir crescimento organizado da plataforma.
-

3. Estrutura Hierárquica da Equipe

3.1 Tech Lead e Infraestrutura

Responsável:

- Arquitetura técnica.
- Revisão de Pull Requests.
- Padronização de código.
- Integração entre módulos.

- Build, lint e performance.
- Infraestrutura PWA e offline.
- Decisões técnicas globais.

Responsável atual:

- Henrique
-

3.2 Núcleo Frontend Principal

Responsável:

- Home.
- Calendário.
- Navegação principal.
- Organização visual inicial.
- Experiência principal do usuário.

Responsável atual:

- Guilherme
-

3.3 Núcleo de Geolocalização

Responsável:

- Mapa.
- Localização.
- Marcadores.
- Filtros geográficos.
- Sellers no mapa.

Responsável atual:

- Flavio
-

3.4 Núcleo Educacional

Responsável:

- Escolinha.
- Lições.
- Progresso do usuário.
- Conteúdo acessível.
- Fluxo offline educacional.

Responsável atual:

- Julia
-

3.5 Núcleo Marketplace

Responsável:

- Catálogo.
- Produtos.
- Cadastro de produtos.
- Upload.
- Pix e pagamentos.
- Fluxo comercial.

Responsável atual:

- Cecilia
-

3.6 Núcleo de Dados e Backend

Responsável:

- Banco de dados.
- Firebase/Supabase.
- Tipagens globais.
- Seeds.
- Services.
- Estrutura de dados.

Responsável atual:

- Marianne
-

4. Estrutura Oficial do Desenvolvimento

Todo desenvolvimento deve seguir as seguintes fases:

FASE 1 — Infraestrutura

Objetivo: Criar a fundação técnica do projeto.

Inclui:

- Estrutura inicial.
 - Rotas.
 - Organização de pastas.
 - Componentes compartilhados.
 - Configuração base.
 - Dados mockados.
 - Tema visual.
 - Base do frontend.
-

FASE 2 — Desenvolvimento Modular

Objetivo: Expandir funcionalidades reais da plataforma.

Inclui:

- Integração entre módulos.
 - Catálogo.
 - Escolinha.
 - Mapa.
 - Cadastro.
 - WhatsApp.
 - Uploads.
 - Pix.
 - Dados reais.
-

FASE 3 — QA e Estabilização

Objetivo: Garantir funcionamento real do sistema.

Inclui:

- Testes reais.
 - Responsividade.
 - Correção de bugs.
 - Acessibilidade.
 - Performance.
 - Offline.
 - Fluxos principais.
-

FASE 4 — Release Final

Objetivo: Preparar o projeto para entrega final.

Inclui:

- Organização do código.
- Revisões finais.
- Fechamento de pendências.
- Documentação.
- Ajustes finais.
- Revisão geral.
- Deploy.

5. Fluxo Oficial de Desenvolvimento

Todos os membros devem seguir o mesmo fluxo operacional.

PASSO 1 — Atualizar Projeto

Antes de iniciar qualquer tarefa:

```
git checkout main  
git pull origin main
```

PASSO 2 — Criar Branch

Cada tarefa deve possuir sua própria branch.

Exemplo:

```
git checkout -b feat/home-banner
```

Nunca desenvolver diretamente na branch main.

PASSO 3 — Desenvolvimento

Durante o desenvolvimento:

- Alterar apenas arquivos da própria área.
 - Evitar mudanças globais sem alinhamento.
 - Usar componentes compartilhados sempre que possível.
 - Comunicar alterações estruturais.
 - Fazer commits pequenos e organizados.
-

PASSO 4 — Testes Locais

Antes de enviar para GitHub:

```
npm run build  
npm run lint
```

Também é obrigatório:

- Testar navegação.
- Testar responsividade.
- Verificar console.
- Confirmar funcionamento principal.

PASSO 5 — Commit e Push

Exemplo:

```
git add .  
git commit -m "feat: adiciona cards da home"  
git push origin feat/home-banner
```

PASSO 6 — Pull Request

Todo Pull Request deve conter:

- O que foi feito.
- Quais telas foram alteradas.
- Como testar.
- Dependências.
- Pendências restantes.

PASSO 7 — Revisão Técnica

Nenhum merge deve acontecer sem revisão.

A revisão técnica deve validar:

- organização
- padrão
- funcionamento
- impacto em outros módulos

6. Organização Oficial do GitHub

Regras obrigatórias

- Nunca alterar a main diretamente.
 - Cada tarefa deve possuir branch própria.
 - Não misturar múltiplos módulos no mesmo PR.
 - Resolver conflitos localmente.
 - Atualizar branch antes do merge.
-

7. Arquivos de Alto Risco

Os seguintes arquivos exigem extrema atenção:

- frontend/src/main.tsx
- frontend/src/routes/router.tsx
- frontend/src/index.css
- frontend/package.json
- frontend/src/shared/

Mudanças nesses arquivos devem ser comunicadas para toda a equipe.

8. Dependências Entre Módulos

Alguns módulos dependem diretamente de outros.

Exemplos:

- Home depende do catálogo.
- Catálogo depende do banco.
- Mapa depende de sellers.
- Escolinha depende das lições.
- Upload depende da infraestrutura.
- Offline depende das rotas organizadas.

Sempre verificar dependências antes de iniciar tarefas.

9. Critérios Oficiais Para Considerar Uma Tarefa Pronta

Uma tarefa só pode ser considerada concluída quando:

- Build funcionando.

- Lint sem erros graves.
 - Navegação funcionando.
 - Responsividade validada.
 - Código documentado.
 - Pull Request criado.
 - Revisão concluída.
-

10. Organização da Comunicação

A equipe deve manter comunicação organizada.

Canais recomendados:

- frontend
 - backend
 - infra
 - bugs
 - pull-requests
 - deploy
 - urgente
-

11. Organização Recomendada da Documentação

Estrutura recomendada:

/docs /arquitetura /api /fluxos /checklists /padroes /deploy /deciso-es-tecnicas

12. Objetivo Final da Operação

A organização foi criada para garantir:

- Desenvolvimento escalável.
- Clareza operacional.
- Colaboração eficiente.
- Redução de conflitos.
- Qualidade técnica.
- Facilidade de manutenção.
- Entregas mais rápidas.
- Organização profissional do projeto.

Todos os membros devem seguir este manual durante toda a evolução da plataforma.